

Consegna 3 - Progettazione (Relazione da Consegnare)

Maria Andrea Zoccoli, 15-05-2026

Ambiente: PC Locale (Intel i7-1165G7, WSL2 Ubuntu, Docker)

Calcolo di Pi Greco (CPI) e Legge di Amdahl

Analisi Algoritmica e Risultati

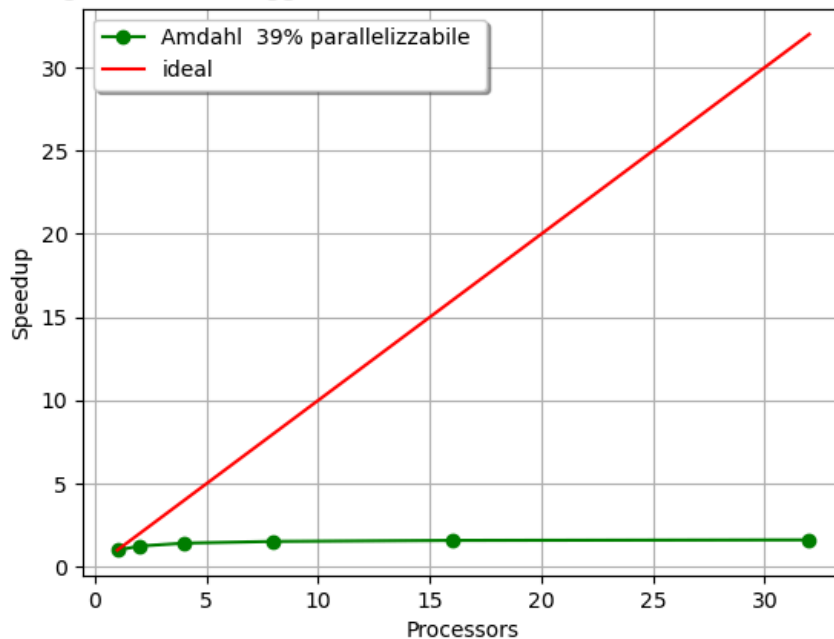
L'obiettivo dell'esercizio è l'approssimazione del valore di π tramite il metodo dei rettangolo applicato a un integrale definito. Il calcolo è intrinsecamente parallelo, poiché ogni iterazione del ciclo `for` è indipendente dalle altre.

Dalle esecuzioni effettuate sul sistema locale, è stato possibile quantificare l'impatto della porzione sequenziale traite l'uso del programma `cpi+amdahl.c`:

- **Tempo Parallelo (t_p):** 0.32420 s
- **Tempo Non Parallelizzabile (t_{np}):** 0.50035 s (ritardo artificiale introdotto con `-s 500000`).
- **Tempo Totale:** 0.82455 s

#Iter	error	tp	tnp	tp1+tnp	hostname
100000000,	6.33271213e-13,	0.32420,	0.50035,	0.82455,	andrea

La frazione di codice parallelizzabile (f) è risultata pari al **39,3%** ($0.32420/0.82455$). Secondo la Legge di Amdahl, lo speedup massimo teorico (S_{max}) per questa configurazione è limitato a **1.64** ($1/(1 - 0.393)$). Come mostrato nel grafico `cpi+amdahl.png`, la curva dello speedup si appiattisce asintoticamente verso questo valore, rendendo l'aggiunta di ulteriori processori (oltre i 4-8 core) inefficace per migliorare le performance.



La linea rossa (ideal) rappresenta uno speedup lineare: se raddoppio i core, il programma va al doppio della velocità. La linea verde (Amdahl) mostra che anche avendo a disposizione 32 processori, lo speedup si appiattisce intorno al valore 1.6x. Perché se il 61% del programma non può essere parallelizzato, non importa quanti core aggiungiamo, quel 61% dovrà sempre essere eseguito da un singolo core, dettando il limite massimo di velocità.

Strategie di Progettazione

- **Partizionamento:** si è optato per una decomposizione di dominio di tipo 1D.
- **Granularità:** per massimizzare il bilanciamento del carico, è preferibile una **grana fine** (suddivisione in piccoli blocchi o *chunks*), che permette ai thread dell'i7 di saturare la pipeline di calcolo evitando tempo di idle causati da eventuali processi di background del sistema operativo.
- **Sincronizzazione:** nel modello a memoria distribuita, l'overhead di comunicazione è trascurabile, richiedendo solo l'invio di $P - 1$ messaggi da 6 byte al termine della riduzione finale.

Propagazione del Calore (HEAT)

Profilazione e Identificazione Hotspot

Il programma `heat.c` simula la diffusione termica su una griglia 2D tramite l'iteratore di Jacobi. Sono state identificate nel codice le funzioni critiche per la gestione delle temperature costanti (`nit_center`, `Init_left`, `Init_top`) e per le condizioni al contorno periodiche (`copy_rows`, `copy_cols`).

La profilazione eseguita con `gprof` su un dominio 4096x4096x500 ha evidenziato la seguente distribuzione del carico computazionale:

```
andre@andrea:heat$ sh heat-pg.slurm
4096, 4096, 500, 20.893, 15.495
Flat profile:
```

Each sample counts as 0.01 seconds.

%	cumulative	self		self	total	
time	seconds	seconds	calls	ms/call	ms/call	name
76.70	17.41	17.41	500	34.82	34.82	Jacobi_Iterator_CPU

22.51	22.52	5.11	500	10.22	10.22	Init_center
0.62	22.66	0.14				main
0.18	22.70	0.04	1	40.00	40.00	print_colormap
0.00	22.70	0.00	1	0.00	0.00	options

Call graph

granularity: each sample hit covers 4 byte(s) for 0.04% of 22.70 seconds

index	% time	self	children	called	name
<spontaneous>					
[1]	100.0	0.14	22.56		main [1]
		17.41	0.00	500/500	Jacobi_Iterator_CPU [2]
		5.11	0.00	500/500	Init_center [3]
		0.04	0.00	1/1	print_colormap [4]
		0.00	0.00	1/1	options [5]

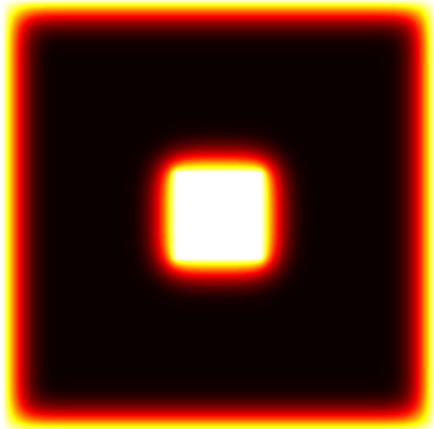
		17.41	0.00	500/500	main [1]
[2]	76.7	17.41	0.00	500	Jacobi_Iterator_CPU [2]

		5.11	0.00	500/500	main [1]
[3]	22.5	5.11	0.00	500	Init_center [3]

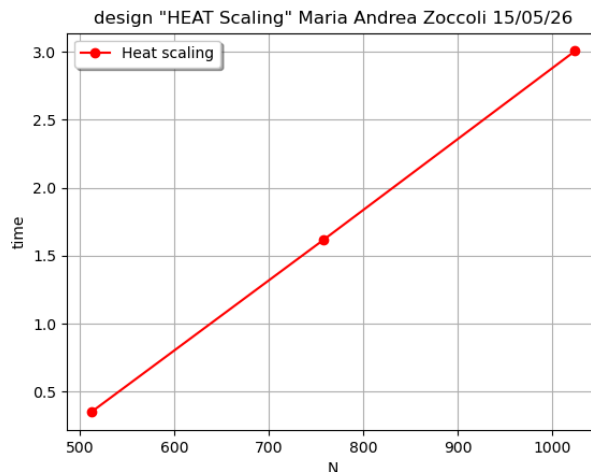
		0.04	0.00	1/1	main [1]
[4]	0.2	0.04	0.00	1	print_colormap [4]

		0.00	0.00	1/1	main [1]
[5]	0.0	0.00	0.00	1	options [5]

Funzione	% Tempo Totale	
Jacobi_Iterator_CPU	78.52%	Hotspot primario: parallelizzabile per righe (decomposizione di dominio).
Init_center	21.01%	Frazione sequenziale critica che limita lo speedup (Amdahl).
copy_cols / rows	<0.1%	Overhead locale trascurabile, ma critico in memoria distribuita.

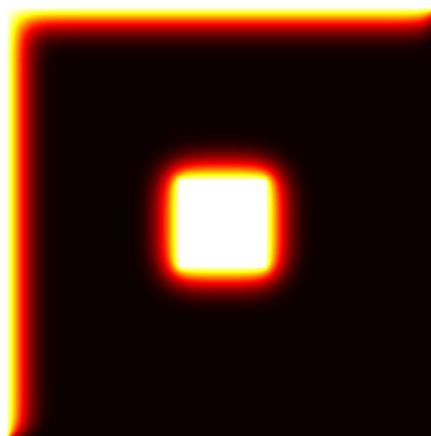


Il grafico dello scaling conferma sperimentalmente questi dati: si osserva chiaramente come, al crescere della dimensione della superficie, il tempo speso nella funzione Jacobi (jtime) tenda a coincidere quasi totalmente con il tempo di esecuzione globale (time), confermandone la centralità per l'ottimizzazione parallela.



Impatto della Parallelizzazione e Comunicazione

La disabilitazione delle condizioni periodiche (commentando `copy_cols` e `copy_rows`) ha dimostrato una variazione nel profilo termico finale:



In un'ottica di parallelizzazione a memoria distribuita, la comunicazione tra i processi è governata dalla formula:

$$T_{comm} = 4 \times Iter \times T_{lat} \times (N/Band)$$

Dati i risultati della profilazione, per mitigare l'overhead di comunicazione è necessaria una **decomposizione a grana grossa** (blocchi contigui di righe), massimizzando il rapporto tra calcolo interno e scambio di dati di bordo (*ghost cells*).

Fattorizzazione RSA

Analisi della Ricerca Brute-Force

L'algoritmo di fattorizzazione opera tramite una ricerca sequenziale di numeri primi che dividono il Modulo RSA. Nell'esecuzione di test su un modulo a 68 bit, la soluzione è stata individuata al blocco 221 in un tempo totale di 1m 27s.

Load Balancing e Gestione della Terminazione

Questo problema presenta sfide uniche rispetto a CPI e HEAT a causa della sua natura non deterministica:

- **Bilanciamento del Carico:** Una decomposizione statica a grana grossa risulterebbe inefficiente. La strategia progettuale ottimale prevede un **bilanciamento dinamico a grana fine** (coda di task), dove ogni thread richiede un nuovo blocco di indirizzi non appena termina il precedente.
- **Early Termination:** È fondamentale implementare un meccanismo di uscita anticipata coordinata. In memoria condivisa, un flag booleano atomico permette la terminazione istantanea di tutti i thread; in memoria distribuita, è necessario un segnale di broadcast per arrestare i nodi del cluster non appena il fattore viene individuato, riducendo al minimo il lavoro inutile.